Mardi 27/10 — Terminal, SSH et poste de développeur

Kevin Bardoux

mardi 27 octobre 2026

Table des matières

Objectifs 2
Matin — Environnement local, ligne de commande & SSH (transfert sécurisé SFTP/SCP) 2
Rappel (5 min) 2
Installer Git (10 min) 3
Apport 1 — Le terminal (1/2) : se repérer (10 min) 3
Se repérer — attention et vérification 5
Apport 1 — Le terminal (2/2) : les fichiers (10 min) 5
Fichiers — attention et vérification 7
Exercice — Rallye terminal (30 min) 7
Apport 2 — SSH : travailler sur un serveur distant (15 min) 8
SSH — attention et vérification 9
Exercice — Première connexion SSH (30 min) 9
Si le temps le permet — `sftp`, puis pourquoi chiffrer ? 10
À retenir — ce matin 10
Débrief 10
Après-midi — TP — « Mon poste de dev » · Explainshell (documentation en anglais) 10
La check-list C1 (1/3) — VS Code, extensions 10
La check-list C1 (2/3) — Terminal, Node et Git 11
La check-list C1 (3/3) — Clé SSH 12
Explainshell (25 min) 15
À retenir — cet après-midi 15
Si le temps le permet — Vos propres commandes 15

Pour aller plus loin 16
Sources 16

> Note
>
> **Compétences visées** : C1 · apprendre en continu (anglais)
> **Livrable** : la check-list « Mon poste de dev » cochée et votre clé SSH publique générée.

Objectifs

- Se déplacer et manipuler des fichiers dans le terminal.
- Se connecter à un serveur en SSH, télécharger avec `scp` (et `sftp`, en Défi).
- Compléter VS Code (extensions) et configurer Git.
- Générer une clé SSH pour GitHub.

Comment se passe la journée. On commence par installer Git, qui fournit sous Windows le terminal **Git Bash**. Puis deux notions : le terminal, en deux temps (se repérer, puis manipuler des fichiers), et SSH. Le transfert avec `sftp` devient un Défi. Chacune suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification ; un encadré « À retenir » termine chaque demi-journée.

Matin — Environnement local, ligne de commande & SSH (transfert sécurisé SFTP/SCP)

Rappel (5 min)

1. Que fait le DNS ?
2. Que signifie un code `404` ?
3. Quel outil relit le code sans le lancer ?

Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

Installer Git (10 min)

Niveau : guidé.

1. Windows : téléchargez **Git for Windows** ; gardez les choix proposés.
2. macOS : dans le Terminal, `xcode-select --install`.
3. Linux : `sudo apt install git`.
4. Ouvrez le terminal (sous Windows : **Git Bash**), tapez `git --version`.
5. **Réussi si** : un numéro de version s'affiche.

Sous Windows, Git for Windows (Git, s. d.-c; Git for Windows, s. d.) installe aussi **Git Bash**, un terminal qui accepte les mêmes commandes que macOS et Linux : c'est celui de la formation, à partir d'aujourd'hui. Lancez-le depuis le menu Démarrer (« Git Bash »). Sous macOS, Apple fournit Git avec les outils en ligne de commande de Xcode (Git, s. d.-b) ; sous Linux, avec le gestionnaire de paquets (Git, s. d.-a).

Vous devez voir, à l'étape 4, sous Windows (le numéro dépend de la version installée ; sous macOS et Linux, la fin `.windows.2` n'apparaît pas) :

```
stagiaire@PC MINGW64 ~
$ git --version
git version 2.56.0.windows.2
```

La première ligne est l'invite de Git Bash : votre nom d'utilisateur, le nom de l'ordinateur, puis le dossier courant (`~`, votre dossier personnel). Vous tapez après le `$`. Un message de Git vous bloque cette semaine ? La fiche Lire un message d'erreur explique les plus fréquents.

Apport 1 — Le terminal (1/2) : se repérer (10 min)

- **Pourquoi** : serveurs, Git, Node se pilotent au clavier.
- **Terminal** : des commandes, tapées dans un dossier.

Pourquoi. Les outils du développeur se pilotent au clavier, et un serveur n'a souvent pas d'écran : il faut savoir s'y déplacer sans souris. Une commande se relance, se copie, s'automatise.

Définition. Le terminal est un programme où l'on tape des commandes ; chaque commande s'exécute dans un dossier, le **dossier courant**. `pwd` l'affiche, `ls` liste son contenu, `cd` change de dossier (MDN Web Docs, s. d.). Sous Windows, on utilise **Git Bash**, installé à l'instant ; sous macOS et Linux, le terminal fourni.

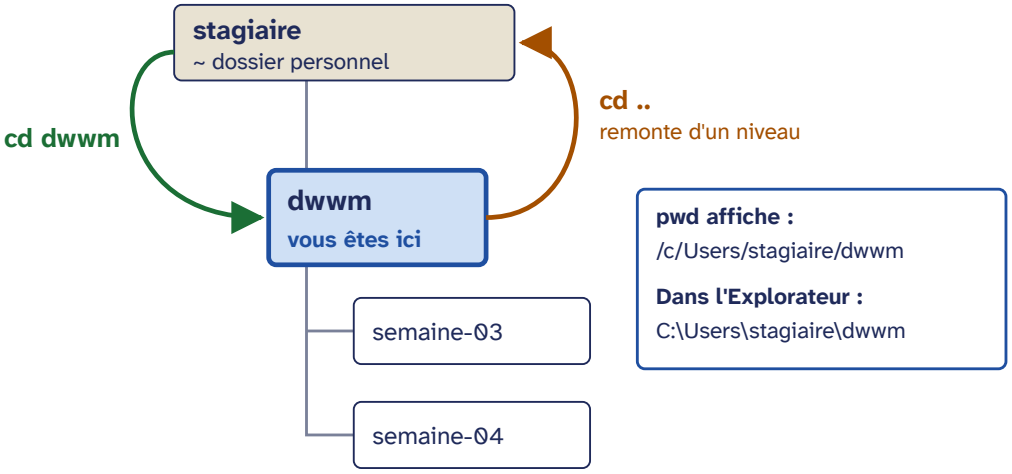


Figure 1. – Les dossiers forment un arbre. `cd dwwm` descend dans un dossier, `cd ..` remonte d'un niveau ; `pwd` affiche le chemin du dossier où vous êtes.

Exemple.

```
pwd                 # où suis-je ?
ls -la              # que contient ce dossier ?
cd dwwm             # entrer dans un dossier
cd ..               # remonter d'un niveau
mkdir essais        # créer un dossier
touch notes.txt     # créer un fichier vide
```

Pas à pas. Le formateur tape chaque commande ; vous la retapez aussitôt.

1. Tapez `pwd` : le chemin de votre dossier personnel s'affiche.
2. Tapez `ls -la` : tout le contenu, fichiers cachés compris (`-a`), en détail (`-l`).
3. Tapez `cd dwwm`, puis `pwd` : le chemin se termine par `dwwm`.
4. Tapez `cd ..`, puis `pwd` : vous êtes remonté d'un niveau.
5. Tapez `cd dw`, puis la touche **Tab** : le nom se complète.

6. Appuyez sur la **flèche du haut** : la commande précédente revient.

Vous devez voir, aux étapes 1, 3 et 4 (avec votre nom d'utilisateur à la place de `stagiaire`) :

```
stagiaire@PC MINGW64 ~
$ pwd
/c/Users/stagiaire

stagiaire@PC MINGW64 ~
$ cd dwwm

stagiaire@PC MINGW64 ~/dwwm
$ pwd
/c/Users/stagiaire/dwwm

stagiaire@PC MINGW64 ~/dwwm
$ cd ..

stagiaire@PC MINGW64 ~
$ pwd
/c/Users/stagiaire
```

Git Bash écrit le disque `C:` sous la forme `/c` ; dans l'invite, `~` désigne votre dossier personnel. `cd` n'affiche rien quand il réussit : l'invite, elle, change.

Se repérer — attention et vérification

> ⚠ Attention
>
> `No such file or directory` : le nom est mal écrit, ou vous n'êtes pas dans le bon dossier. Tapez `pwd`, puis `ls`, avant de réessayer.

- **Vérifiez** : depuis `dwwm/semaine-03`, où mène `cd ..` ?

Apport 1 — Le terminal (2/2) : les fichiers (10 min)

- **Pourquoi** : copier, renommer, chercher sans quitter le clavier.
- **`cp`, `mv`, `rm`, `cat`, `grep`** : copier, déplacer, supprimer, lire, chercher.

Pourquoi. Sur un serveur, sans explorateur de fichiers, il faut copier, renommer, lire et chercher avec des commandes.

Définition. `cp a.txt b.txt` copie ; `mv a.txt c.txt` déplace ou renomme ; `rm fichier` supprime ; `cat fichier` affiche le contenu ; `grep mot fichier` cherche un mot ; `commande --help` affiche l'aide, en anglais.

Exemple.

```
echo Ada > prenom.txt          # créer un fichier qui
contient Ada
cp prenom.txt copie.txt        # le copier
mv copie.txt sauvegarde.txt    # renommer la copie
cat sauvegarde.txt             # afficher : Ada
grep Ada prenom.txt            # chercher : Ada
rm sauvegarde.txt              # supprimer
```

Pas à pas. Dans un dossier d'essai (`mkdir essais`, puis `cd essais`), le formateur tape chaque ligne ; vous la retapez aussitôt, puis vous tapez `ls` pour voir l'effet.

Vous devez voir :

```
stagiaire@PC MINGW64 ~/essais
$ echo Ada > prenom.txt

stagiaire@PC MINGW64 ~/essais
$ cp prenom.txt copie.txt

stagiaire@PC MINGW64 ~/essais
$ ls
copie.txt  prenom.txt

stagiaire@PC MINGW64 ~/essais
$ mv copie.txt sauvegarde.txt

stagiaire@PC MINGW64 ~/essais
$ ls
prenom.txt  sauvegarde.txt
```

```
stagiaire@PC MINGW64 ~/essais
$ cat sauvegarde.txt
Ada

stagiaire@PC MINGW64 ~/essais
$ grep Ada prenom.txt
Ada

stagiaire@PC MINGW64 ~/essais
$ rm sauvegarde.txt

stagiaire@PC MINGW64 ~/essais
$ ls
prenom.txt
```

`cp`, `mv` et `rm` ne disent rien quand ils réussissent : `ls` montre leur effet. `rm` ne demande pas de confirmation, et il n'y a pas de corbeille.

Fichiers — attention et vérification

> ⚠ Attention
>
> `rm` supprime **définitivement** : pas de corbeille. Relisez la commande avant Entrée, et faites `ls` après.

- **Vérifiez** : quelle commande renomme `a.txt` en `b.txt` ?

Exercice — Rallye terminal (30 min)

Niveau : semi-guidé.

1. Ensemble (5 min) : les étapes 1 et 2.
2. Seuls : les étapes 3 à 10, au clavier uniquement.
3. Notez chaque commande tapée sur votre fiche.
4. **Réussi si** : l'étape 10 montre que `copie.txt` a disparu.
5. Allez dans votre dossier `dwwm` et affichez son chemin complet.
6. Créez un dossier `semaine-04`, puis entrez dedans.
7. Créez les dossiers `rallye/a` et `rallye/b` (indice : `mkdir -p`).

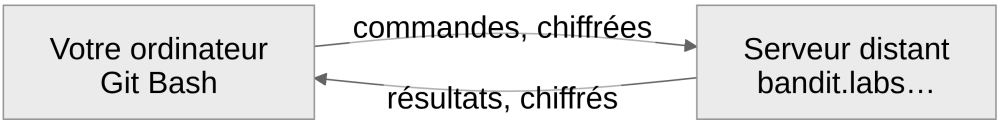
8. Créez un fichier `rallye/a/depart.txt` contenant votre prénom : `echo Ada > rallye/a/depart.txt`.
9. Copiez-le dans `rallye/b` sous le nom `copie.txt`.
10. Renommez `rallye/a/depart.txt` en `rallye/a/arrivee.txt`.
11. Affichez le contenu de `rallye/b/copie.txt`.
12. Listez tout le contenu de `rallye`, sous-dossiers compris (`ls -R`).
13. Cherchez votre prénom dans tout le dossier `rallye` (`grep -r`).
14. Supprimez `rallye/b/copie.txt`, puis vérifiez qu'il a disparu.

Apport 2 — SSH : travailler sur un serveur distant (15 min)

- **Pourquoi** : agir sur un serveur à distance, en toute sécurité.
- **SSH** : un terminal chiffré sur une autre machine.

Pourquoi. Un site en ligne tourne sur un serveur, loin de vous. Pour y travailler, il faut s'y connecter, et que personne ne puisse lire ce qui passe sur le réseau.

Définition. SSH ouvre un terminal chiffré sur une machine distante (OpenBSD, s. d.-c) ; `scp` copie des fichiers par SSH (OpenBSD, s. d.-a) ; `sftp` ouvre une session interactive de transfert (`get` pour télécharger, `put` pour envoyer) (OpenBSD, s. d.-b). On s'entraîne sur les serveurs du jeu Bandit : l'utilisateur `bandit0`, le mot de passe `bandit0` et le port 2220 sont indiqués sur la page du niveau 0 (OverTheWire, s. d.).



Avec SSH, vous tapez sur votre ordinateur, les commandes s'exécutent sur le serveur ; ce qui passe sur le réseau est chiffré.

Exemple.

```
ssh bandit0@bandit.labs.overthewire.org -p 2220
scp -P 2220
```

```
bandit0@bandit.labs.overthewire.org:readme .
sftp -P 2220 bandit0@bandit.labs.overthewire.org
```

Pas à pas. Le formateur fait chaque étape au vidéoprojecteur ; vous la refaites aussitôt.

1. Tapez la commande `ssh` de la ligne 1. À la première connexion, répondez `yes` pour accepter le serveur.
2. Tapez le mot de passe `bandit0` : rien ne s'affiche pendant la frappe, c'est normal. Entrée.
3. L'invite change : vous êtes sur le serveur. Tapez `ls`, puis `exit` pour revenir sur votre ordinateur.
4. Tapez la commande `scp` de la ligne 2 : le fichier `readme` est copié dans votre dossier courant (le `.` final).
5. Tapez `ls` : `readme` est là.

SSH — attention et vérification

> ⚠ Attention
>
> Le port s'écrit `-p` pour `ssh`, mais `-P` (majuscule) pour `scp` et `sftp`. Une connexion refusée vient souvent de là.

- **Vérifiez** : quelle commande quitte le serveur ?

Exercice — Première connexion SSH (30 min)

Niveau : guidé.

1. Ensemble (5 min) : la connexion `ssh` au niveau `0` de Bandit.
2. Seuls : listez les fichiers, affichez `readme`, puis `exit`.
3. Téléchargez `readme` avec `scp`.
4. **Réussi si** : `ls` montre `readme` dans votre dossier.

Le fichier `readme` contient le mot de passe du niveau suivant : vous en aurez besoin demain. Ne le publiez nulle part : le jeu n'a de sens que si chacun cherche par lui-même.

Si le temps le permet — `sftp`, puis pourquoi chiffrer ?

> **Défi**
>
> Téléchargez aussi `readme` avec `sftp`, la session interactive de transfert : `sftp -P 2220 bandit0@bandit.labs.overthewire.org`, puis `get readme`, puis `exit`. **Réussi si** : `ls` montre `readme` dans votre dossier.

1. Demandez à CS50.ai : « Quelle différence entre SSH et Telnet ? »
2. Vérifiez qu'il vous guide sans répondre à votre place.

À retenir — ce matin

> **À retenir**
>
> - **Se repérer** : `pwd`, `ls`, `cd` ; **Tab** complète les noms.
> - **Fichiers** : `cp`, `mv`, `cat`, `grep` ; `rm` est définitif.
> - **SSH** : `ssh -p`, mais `scp -P` et `sftp -P` ; `exit` pour sortir.

Le détail, notion par notion :

- **Se repérer** : chaque commande s'exécute dans le dossier courant ; `pwd` le montre, `ls -la` liste tout, `cd ..` remonte ; Tab complète, la flèche du haut rappelle.
- **Fichiers** : `cp` copie, `mv` déplace ou renomme, `cat` affiche, `grep` cherche ; `rm` supprime sans corbeille ; `--help` donne l'aide.
- **SSH** : un terminal chiffré sur un serveur ; `scp` et `sftp` transfèrent des fichiers ; le port s'écrit `-p` ou `-P` selon la commande.

Débrief

- Quelle commande pour savoir où l'on se trouve ?
- Pourquoi `rm` est-il dangereux ?
- À quoi sert le port dans une commande SSH ?

Après-midi — TP — « Mon poste de dev » · Explainshell (documentation en anglais)

La check-list C1 (1/3) — VS Code, extensions

Niveau : guidé.

- □ Dossier ouvert ; extensions recommandées installées
- □ VS Code en français (pack de langue), après redémarrage
- □ `chat.disableAIFeatures` toujours coché
- □ ESLint souligne les erreurs dans l'éditeur

VS Code est installé depuis la semaine 2. Aujourd'hui, on le complète :

1. Téléchargez l'archive de la semaine ; extrayez-la dans `dwwm/semaine-04`.
2. Dans VS Code : *File*, *Open Folder...*, puis `dwwm/semaine-04`.
3. VS Code propose d'installer les extensions recommandées : acceptez. Le fichier `.vscode/extensions.json` du dossier recommande ESLint, le pack de langue français et Live Preview (pour la semaine 5) (Microsoft, s. d.-a).
4. Redémarrez VS Code quand il le demande : l'interface passe en français.
5. Ouvrez les paramètres (`Ctrl+,` (windows), `Cmd+,` (mac)) ; cherchez `chat.disableAIFeatures` ; vérifiez que la case est toujours cochée (Microsoft, s. d.-b). La charte interdit l'autocomplétion par IA jusqu'au palier 2b (semaine 16).
6. Ouvrez un fichier `.js` du dossier et ajoutez `var x = 1;` : ESLint le souligne. Annulez la modification.

La check-list C1 (2/3) — Terminal, Node et Git

- □ Windows : Git Bash est le terminal par défaut de VS Code
- □ Dans le terminal de VS Code : `node -v` affiche une version LTS
- □ `git --version` affiche une version
- □ `git config --global` : nom, e-mail, branche `main`

Git est installé depuis ce matin. Sous Windows, dans VS Code, lancez la commande **Terminal: Select Default Profile** et choisissez **Git Bash** : le réglage `terminal.integrated.defaultProfile.windows` est alors enregistré (Microsoft, s. d.-c). Ouvrez un nouveau terminal Git Bash et tapez `node -v` : Node.js, installé en semaine 2, doit répondre. Sinon, fermez puis rouvrez VS Code et réessayez ; s'il ne répond

toujours pas, appelez le formateur. Configurez ensuite votre identité Git (Scott Chacon et Ben Straub, s. d.) :

```
git config --global user.name "Prénom Nom"
git config --global user.email prenom.nom@exemple.fr
git config --global init.defaultBranch main
```

Utilisez l'adresse e-mail de votre compte GitHub, créé lundi. La dernière ligne nomme `main` la branche principale de vos futurs dépôts, comme sur GitHub.

Vous devez voir : rien après chacune des trois commandes. Pas de message : tout s'est bien passé. Pour vérifier, tapez `git config --global user.name` : votre nom s'affiche.

```
stagiaire@PC MINGW64 ~
$ git config --global user.name "Prénom Nom"

stagiaire@PC MINGW64 ~
$ git config --global user.email prenom.nom@exemple.fr

stagiaire@PC MINGW64 ~
$ git config --global init.defaultBranch main

stagiaire@PC MINGW64 ~
$ git config --global user.name
Prénom Nom
```

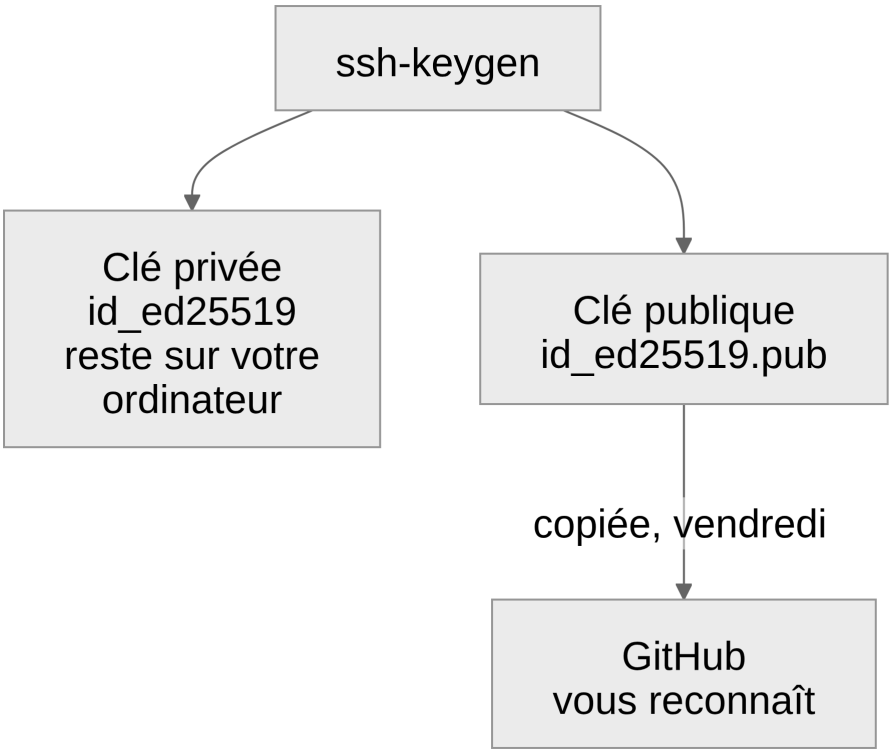
La check-list C1 (3/3) — Clé SSH

- □ Une paire de clés générée avec `ssh-keygen`
- □ Une phrase de passe choisie pour protéger la clé
- □ Vous savez où est la clé **publique** (`.pub`)
- □ La clé **privée** ne quitte jamais votre poste

Commande recommandée par GitHub, avec votre adresse e-mail (GitHub, s. d.) :

```
ssh-keygen -t ed25519 -C "prenom.nom@exemple.fr"
```

Acceptez l'emplacement proposé ; la clé publique est le fichier qui se termine par `.pub`. Une phrase de passe protège la clé : elle vous sera demandée quand vous l'utiliserez (GitHub, s. d.; OpenBSD, s. d.-d). Vous ajouterez la clé publique à votre compte GitHub vendredi.



La clé privée reste sur votre ordinateur ; la clé publique est déposée sur GitHub (vendredi), qui s'en sert pour vous reconnaître.

Vous devez voir (votre empreinte, après `SHA256:`, et le dessin seront différents ; `ssh-keygen` s'affiche en anglais) :

```
stagiaire@PC MINGW64 ~
$ ssh-keygen -t ed25519 -C "prenom.nom@exemple.fr"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/
stagiaire/.ssh/id_ed25519):
Created directory '/c/Users/stagiaire/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/
stagiaire/.ssh/id_ed25519
Your public key has been saved in /c/Users/
```

```
stagiaire/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:CylUX/2idkBz6+0oP3rntvqr+Vk52PjJUTcTBEJMKjM
prenom.nom@exemple.fr
The key's randomart image is:
+--[ED25519 256]--+
|      .   == ... |
|     . . .+.+ .  |
|    .  E.o o o . |
|   .    .+ . o . .|
|    . o S  + o oo|
|     . . .o o = *|
|        .. . = =.|
|          . +.*o+|
|          .==XB* |
+----[SHA256]-----+
```

Lisez-la ligne par ligne :

1. `Enter file…` : appuyez sur Entrée, sans rien taper, pour garder l'emplacement proposé.
2. `Created directory…` : le dossier `.ssh` vient d'être créé. Cette ligne n'apparaît que s'il n'existait pas.
3. `Enter passphrase…`, puis `Enter same passphrase again` : tapez votre phrase de passe deux fois. Rien ne s'affiche pendant la frappe : c'est normal.
4. `saved in …id_ed25519` : la clé **privée** ; `saved in … id_ed25519.pub` : la clé **publique**.
5. Le dessin (*randomart*) sert seulement à comparer deux clés d'un coup d'œil : rien à noter.

Puis vérifiez que les deux fichiers sont là :

```
stagiaire@PC MINGW64 ~
$ ls ~/.ssh
id_ed25519  id_ed25519.pub
```

Si une clé existe déjà, `ssh-keygen` affiche `…/id_ed25519 already exists.`, puis `Overwrite (y/n)?` : répondez `n`, puis appelez le formateur. Écraser la clé couperait les accès qu'elle ouvre déjà.

Explainshell (25 min)

Niveau : semi-guidé.

1. Ouvrez Explainshell ; collez la première commande (ci-dessous).
2. Pour chaque option, notez en français ce qu'elle fait.
3. Recommencez avec les trois autres commandes.
4. **Réussi si** : quatre commandes expliquées avec vos mots.

Explainshell fait correspondre chaque argument d'une commande à son explication dans la documentation (explainshell.com, s. d.). Commandes à expliquer :

1. `ls -la`
2. `grep -r "code" .`
3. `find . -name "*.txt"`
4. `ssh bandit0@bandit.labs.overthewire.org -p 2220`

À retenir — cet après-midi

> À retenir
>
> - **VS Code** : extensions recommandées, en français, IA toujours désactivée.
> - **Git** : `git config --global` une fois : nom, e-mail, `main`.
> - **Clé SSH** : la `.pub` se partage ; la clé privée, jamais.

Critères de réussite

- □ Les trois parties de la check-list C1 sont cochées
- □ La connexion SSH à Bandit et les deux téléchargements ont réussi
- □ Les quatre commandes sont expliquées avec vos mots

Si le temps le permet — Vos propres commandes

1. Collez dans Explainshell trois commandes utilisées ce matin.
2. Notez une option que vous ne connaissiez pas.

Pour aller plus loin

- `ls --help` : trouvez l'option qui trie par taille.
- Et celle qui affiche les tailles en Ko, Mo ?

Sources

explainshell.com. s. d. « explainshell.com - match command-line arguments to their help text ». Consulté le 26 septembre 2026. https://explainshell.com/.

Git. s. d.-a. « Git - Install for Linux ». Consulté le 3 octobre 2026. https://git-scm.com/install/linux.

Git. s. d.-b. « Git - Install for macOS ». Consulté le 3 octobre 2026. https://git-scm.com/install/mac.

Git. s. d.-c. « Git - Install for Windows ». Consulté le 26 septembre 2026. https://git-scm.com/install/windows.

Git for Windows. s. d. « Git for Windows ». Consulté le 26 septembre 2026. https://gitforwindows.org/.

GitHub. s. d. « Generating a new SSH key and adding it to the ssh-agent ». Consulté le 26 septembre 2026. https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent.

MDN Web Docs. s. d. « Cours express sur l'invite de commande - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Getting_started/Environment_setup/Command_line.

Microsoft. s. d.-a. « Extension Marketplace - Visual Studio Code ». Consulté le 26 septembre 2026. https://code.visualstudio.com/docs/configure/extensions/extension-marketplace.

Microsoft. s. d.-b. « Set up GitHub Copilot in VS Code (Remove AI features) ». Consulté le 26 septembre 2026. https://code.visualstudio.com/docs/copilot/setup.

Microsoft. s. d.-c. « Terminal Profiles - Visual Studio Code ». Consulté le 26 septembre 2026. https://code.visualstudio.com/docs/terminal/profiles.

OpenBSD. s. d.-a. « scp(1) - OpenBSD manual pages ». Consulté le 26 septembre 2026. https://man.openbsd.org/scp.

OpenBSD. s. d.-b. « sftp(1) - OpenBSD manual pages ». Consulté le 26 septembre 2026. https://man.openbsd.org/sftp.

OpenBSD. s. d.-c. « ssh(1) - OpenBSD manual pages ». Consulté le 26 septembre 2026. https://man.openbsd.org/ssh.

OpenBSD. s. d.-d. « ssh-keygen(1) - OpenBSD manual pages ». Consulté le 26 septembre 2026. https://man.openbsd.org/ssh-keygen.

OverTheWire. s. d. « Bandit Level 0 ». Consulté le 26 septembre 2026. https://overthewire.org/wargames/bandit/bandit0.html.

Scott Chacon et Ben Straub. s. d. « Pro Git : Paramétrage à la première utilisation de Git ». Consulté le 26 septembre 2026. https://git-scm.com/book/fr/v2/D%C3%A9marrage-rapide-Param%C3%A9trage-%C3%A0-la-premi%C3%A8re-utilisation-de-Git.